

Problem Statements Report

Generated on: 2026-03-08 22:09

Week 4: Stacks and Queues

Q1. Parenthesis Matching

MEDIUM CODING

Implement a C program using a Stack to check if a given expression has balanced parentheses. The expression will contain (), {}, and [].

Input Format: A single string containing the expression.

Output Format: Print 'Balanced' if the parentheses are matched, otherwise print 'Not Balanced'.

Constraints: $1 \leq \text{Length of string} \leq 1000$. String contains only parentheses characters.

Hints: Use a stack to track opening brackets. Pop when a closing bracket is encountered and check for a match.

Test Cases:

Sample Test Case #1

Input:

()

Expected Output:

Balanced

Sample Test Case #2

Input:

()()

Expected Output:

Balanced

Sample Test Case #3

Input:

((

Expected Output:

Not Balanced

Solution Strategy:

A stack efficiently tracks the latest unmatched opening bracket. Whenever a closing bracket is encountered, it must match the type of the bracket currently at the top of the stack. If types differ or the stack is empty prematurely, or if brackets remain at the end, it's unbalanced.

Reference Solution:

```
#include <stdio.h>
#include <string.h>
int main() {
    char exp[1001], stack[1001];
    int top = -1, i = 0, valid = 1;
    if(scanf("%1000s", exp) != 1) return 0;
    while(exp[i]) {
        if(exp[i] == '(' || exp[i] == '{' || exp[i] == '[') {
            stack[++top] = exp[i];
        } else if(exp[i] == ')' || exp[i] == '}' || exp[i] == ']') {
            if(top == -1) { valid = 0; break; }
            if(exp[i] == ')' && stack[top] != '(') { valid = 0; break; }
            if(exp[i] == '}' && stack[top] != '{') { valid = 0; break; }
            if(exp[i] == ']' && stack[top] != '[') { valid = 0; break; }
            top--;
        }
        i++;
    }
    if(top != -1) valid = 0;
    if(valid) printf("Balanced");
    else printf("Not Balanced");
    return 0;
}
```

Q2. Circular Queue using Array

MEDIUM CODING

Implement a Circular Queue using an array. Support operations: Enqueue, Dequeue, and Display. Proper wrap-around should be handled using modulo arithmetic.

Input Format: Integer S (size of queue) and N (number of operations), followed by N lines of commands: 'enqueue X', 'dequeue', 'display'.

Output Format: Print the result of each operation. For dequeue, print the removed element. Print 'Queue is Full' or 'Queue is Empty' on overflow/underflow. For display, print elements separated by space, or 'Empty'.

Constraints: $1 \leq S \leq 100$, $1 \leq N \leq 1000$.

Hints: Use front and rear initially set to -1. Calculate next position as $(rear + 1) \% S$. It's full if this next position equals front.

Test Cases:

Sample Test Case #1

Input:

```
3 3
enqueue 10
enqueue 20
display
```

Expected Output:

```
10 20
```

Sample Test Case #2

Input:

```
2 4
enqueue 1
enqueue 2
enqueue 3
display
```

Expected Output:

```
Queue is Full
1 2
```

Sample Test Case #3

Input:

```
3 2
dequeue
display
```

Expected Output:

```
Queue is Empty
Empty
```

Solution Strategy:

Circular Queue optimizes space usage by wrapping around using modulo arithmetic (`%`). The critical checks are 'Queue Full' condition `(rear + 1) % S == front` and 'Queue Empty' condition `front == -1`.

Reference Solution:

```

#include <stdio.h>
#include <string.h>
int queue[100], front = -1, rear = -1, size;
void enqueue(int v) {
    if ((rear + 1) % size == front) printf("Queue is Full\n");
    else {
        if (front == -1) front = 0;
        rear = (rear + 1) % size;
        queue[rear] = v;
    }
}
void dequeue() {
    if (front == -1) printf("Queue is Empty\n");
    else {
        printf("%d\n", queue[front]);
        if (front == rear) { front = -1; rear = -1; }
        else front = (front + 1) % size;
    }
}
void display() {
    if (front == -1) printf("Empty\n");
    else {
        int i = front;
        while (1) {
            printf("%d%c", queue[i], i == rear ? '\n' : ' ');
            if (i == rear) break;
            i = (i + 1) % size;
        }
    }
}
int main() {
    int ops, v;
    char op[20];
    if (scanf("%d %d", &size, &ops) != 2) return 0;
    while (ops--) {
        scanf("%s", op);
        if (!strcmp(op, "enqueue")) { scanf("%d", &v); enqueue(v); }
        else if (!strcmp(op, "dequeue")) dequeue();
        else if (!strcmp(op, "display")) display();
    }
    return 0;
}

```

Week 5: Singly Linked List

Q1. Creation

EASY CODING

Implement a C program to Create a Singly Linked List by reading N integer values and constructing nodes dynamically using malloc. After creation, display the list to verify correctness.

Input Format: Integer N (number of elements), followed by N integer values to insert into the list sequentially.

Output Format: Print space-separated elements from head to tail. Print 'Empty' if N is 0.

Constraints: $0 \leq N \leq 1000$. Nodes must be dynamically allocated using malloc.

Hints: Maintain a head pointer initialized to NULL. For each value, create a new node and link it to the end of the list by traversing to the last node.

Test Cases:

Sample Test Case #1

Input:

3
10 20 30

Expected Output:

10 20 30

Sample Test Case #2

Input:

0

Expected Output:

Empty

Sample Test Case #3

Input:

1
99

Expected Output:

99

Solution Strategy:

Creation involves reading N values and dynamically allocating a new node for each using malloc. Each node is linked sequentially via the next pointer. Traversal from head to NULL displays all elements in $O(N)$ time.

Reference Solution:

```

#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;
void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d,
    t->n?' ':'\n'); t=t->n; } } }
int main() {
    int N, v;
    if(scanf("%d", &N) != 1) return 0;
    while(N--) { scanf("%d", &v); ie(v); }
    disp();
    return 0;
}

```

Q2. Insertion Operations

MEDIUM CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Insert at Beginning, Insert at Middle (given position), Insert at End.

- Input Format:** Integer N (number of operations), followed by N lines of commands: 'ins_beg X' (insert at beginning), 'ins_end X' (insert at end), 'ins_pos P X' (insert at position P), 'display' (print list).
- Output Format:** No direct output for insertion. Use 'display' to print space-separated elements or 'Empty' if list is empty.
- Constraints:** $0 \leq N \leq 1000$. Nodes must be dynamically allocated.
- Hints:** Insert at beginning: Create a new node and point it to current head, then update head. Insert at end: Traverse to the last node and link the new node. Insert at middle (position P): Navigate to position P-1 and reroute the 'next' pointer.

Test Cases:

Sample Test Case #1

Input:

```

5
ins_end 10
ins_beg 5
ins_end 20
ins_pos 2 8
display

```

Expected Output:

```

5 8 10 20

```

Sample Test Case #2

Input:

```
2
ins_beg 10
display
```

Expected Output:

```
10
```

Sample Test Case #3

Input:

```
3
ins_end 10
ins_pos 1 5
display
```

Expected Output:

```
5 10
```

Solution Strategy:

Insert at beginning is $O(1)$ by pointing new node to head. Insert at end is $O(N)$ requiring traversal to the last node. Insert at middle (position) is $O(N)$ requiring traversal to position $P-1$ and rerouting the next pointer.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=head; head=nn; }
void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void ip(int p, int v) { if(p==1) ib(v); else { Node* t=head; for(int i=1; i<p-1 && t; i++) t=t->n; if(t) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=t->n; t->n=nn; } } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d, t->n?' ':'\n'); t=t->n; } } }

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "ins_pos")) { scanf("%d %d", &p, &v); ip(p, v); }
        else if(!strcmp(op, "display")) disp();
    }
    return 0;
}
```

Q3. Deletion Operations

MEDIUM

CODING

Implement a C program utilizing Singly Linked Lists focused on the following tasks: Delete at Beginning, Delete at Middle (given position), Delete at End.

Input Format: Integer N (number of operations), followed by N lines of commands: 'ins_end X' (insert at end for setup), 'del_beg' (delete first node), 'del_end' (delete last node), 'del_pos P' (delete node at position P), 'display' (print list).

Output Format: Print 'Underflow' if attempting deletion from an empty list. Use 'display' to print space-separated elements or 'Empty' if list is empty.

Constraints: $0 \leq N \leq 1000$. Nodes must be dynamically allocated.

Hints: Delete at beginning: Update head to head->next and free the old head. Delete at end: Traverse to the second-to-last node and free the last node. Delete at middle (position P): Navigate to position P-1 and reroute the next pointer, then free the removed node.

Test Cases:

Sample Test Case #1

Input:

```
6
ins_end 10
ins_end 20
ins_end 30
del_beg
del_end
display
```

Expected Output:

```
20
```

Sample Test Case #2

Input:

```
2
del_beg
display
```

Expected Output:

```
Underflow
Empty
```

Sample Test Case #3

Input:

```
3
ins_end 10
del_pos 1
display
```

Expected Output:

```
Empty
```


Solution Strategy:

Delete at beginning is $O(1)$ by updating head to head->next and freeing the old node. Delete at end is $O(N)$ requiring traversal to the second-to-last node. Delete at middle (position) is $O(N)$ requiring traversal to position P-1 and rerouting the next pointer.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void db() { if(head) { Node* t=head; head=head->n; free(t); } else printf("Underflow\n"); }
void de() { if(!head) printf("Underflow\n"); else if(!head->n) { free(head); head=NULL; } else
    { Node* t=head; while(t->n->n) t=t->n; free(t->n); t->n=NULL; } }
void dp(int p) { if(!head) printf("Underflow\n"); else if(p==1) db(); else { Node* t=head; for(
    int i=1; i<p-1 && t->n; i++) t=t->n; if(t->n) { Node* tmp=t->n; t->n=tmp->n; free(tmp); } e
lse printf("Underflow\n"); } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d,
    t->n?' ':'\n'); t=t->n; } } }

int main() {
    int N, v, p; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "del_pos")) { scanf("%d", &p); dp(p); }
        else if(!strcmp(op, "display")) disp();
    }
    return 0;
}
```

Q4. Display

EASY CODING

Implement a C program utilizing Singly Linked Lists focused on the following task: Display all elements of a Singly Linked List from head to tail after performing a series of insertions and deletions.

Input Format: Integer N (number of operations), followed by N lines of commands: 'ins_end X' (insert at end), 'ins_beg X' (insert at beginning), 'del_beg' (delete first node), 'del_end' (delete last node), 'display' (print list).

Output Format: For 'display', print space-separated elements from head to tail. Print 'Empty' if the list has no nodes. Print 'Underflow' for deletion on an empty list.

Constraints: $0 \leq N \leq 1000$. Nodes must be dynamically allocated.

Hints: Traverse from head to the last node printing each value. Use a space between elements and a newline after the last element. Handle the empty list case by checking if head is NULL.

Test Cases:

Sample Test Case #1

Input:

```
3
ins_end 10
ins_end 20
display
```

Expected Output:

```
10 20
```

Sample Test Case #2

Input:

```
1
display
```

Expected Output:

```
Empty
```

Sample Test Case #3

Input:

```
4
ins_end 5
ins_end 15
ins_end 25
display
```

Expected Output:

```
5 15 25
```

Solution Strategy:

Display traverses from head to the last node in $O(N)$ time, printing each element. The key is handling the output format correctly - spaces between elements and a newline after the last element. An empty list ($\text{head} == \text{NULL}$) outputs 'Empty'.

Reference Solution:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node* n; } Node;
Node* head = NULL;

void ib(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=head; head=nn; }
void ie(int v) { Node* nn = malloc(sizeof(Node)); nn->d=v; nn->n=NULL; if(!head) head=nn; else
    { Node* t=head; while(t->n) t=t->n; t->n=nn; } }
void db() { if(head) { Node* t=head; head=head->n; free(t); } else printf("Underflow\n"); }
void de() { if(!head) printf("Underflow\n"); else if(!head->n) { free(head); head=NULL; } else
    { Node* t=head; while(t->n->n) t=t->n; free(t->n); t->n=NULL; } }
void disp() { if(!head) printf("Empty\n"); else { Node* t=head; while(t) { printf("%d%c", t->d,
    t->n?' ':'\n'); t=t->n; } } }

int main() {
    int N, v; char op[20];
    if(scanf("%d", &N) != 1) return 0;
    while(N--) {
        scanf("%s", op);
        if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); }
        else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); }
        else if(!strcmp(op, "del_beg")) db();
        else if(!strcmp(op, "del_end")) de();
        else if(!strcmp(op, "display")) disp();
    }
    return 0;
}

```

Week 6: Stack and Queue using Linked List

Q1. Stack Operations using Linked List

MEDIUM CODING

Build a **Stack** dynamically utilizing explicit Linked List principles where items mapping strictly `push`, `pop`, and returning the topmost element seamlessly via `peek` function handlers accurately demonstrate Last-In-First-Out (LIFO) flows.

Input Format: Integer N mapping iterative string blocks calling `push X`, `pop`, `peek`, or `display` stack contents natively.

Output Format: Integer strings evaluating variables returned manually bounding constraints. `Underflow` returned for `pop` issues and `Empty` for missing node visualizations.

Constraints: $N \leq 1000$. Data values within integer limits. Linked List must dynamically adapt scale memory perfectly.

Hints: Visualize a Singly Linked List where you universally insert *AND* delete from exactly the same location (the global `head` pointer, aliased as `top`).

Test Cases:

Sample Test Case #1

Input:

```
4
push 5
push 10
peek
display
```

Expected Output:

```
10
10 5
```

Sample Test Case #2

Input:

```
2
pop
peek
```

Expected Output:

```
Underflow
Empty
```

Sample Test Case #3

Input:

```
3
push 1
pop
peek
```

Expected Output:

```
Empty
```

Solution Strategy:

LIFO (Last In First Out) operations natively emulate inserting uniformly backwards. By setting `nn->next = top`` and `top = nn``, `push`` maintains native runtime bounds seamlessly pushing newest data elements highest logically for absolute minimal `O(1)`` pointer manipulation sequences.

Reference Solution:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* top = NULL;

void push(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->next = top;
    top = nn;
}

void pop() {
    if (!top) { printf("Underflow\n"); return; }
    Node* t = top;
    top = top->next;
    free(t);
}

void peek() {
    if (!top) { printf("Empty\n"); return; }
    printf("%d\n", top->data);
}

void display() {
    if (!top) { printf("Empty\n"); return; }
    Node* t = top;
    while (t) {
        printf("%d%c", t->data, t->next ? ' ' : '\n');
        t = t->next;
    }
}

int main() {
    int N, v;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%s", op);
        if (!strcmp(op, "push")) { scanf("%d", &v); push(v); }
        else if (!strcmp(op, "pop")) pop();
        else if (!strcmp(op, "peek")) peek();
        else if (!strcmp(op, "display")) display();
    }
    return 0;
}

```

Q2. Queue Operations using Linked List

MEDIUM

CODING

Construct a dynamic **Queue** architecture handling iterative node linkages applying First-In-First-Out (FIFO) standards using exact ``enqueue`` natively linked structures and exact ``dequeue`` bounds evaluation algorithms alongside raw state visualization strings.

- Input Format:** Integer N mapped specifically applying logic command routines (``enqueue X``, ``dequeue``, ``front``, ``display``).
- Output Format:** Printed variables verifying space outputs properly, utilizing exact single string prints ("Empty" or "Underflow") to manage pointer collisions.
- Constraints:** $N \leq 1000$. Operations must track bidirectional references gracefully via Dual Pointer Logic.
- Hints:** Keep active logical track of both ``front`` and ``rear`` pointers mapping to distinct unique list extremities resolving dynamic insertion memory allocation issues.

Test Cases:

Sample Test Case #1
Input: 4 enqueue 10 enqueue 20 front display Expected Output: 10 10 20
Sample Test Case #2
Input: 3 enqueue 5 dequeue front Expected Output: Empty
Sample Test Case #3
Input: 2 dequeue display Expected Output: Underflow Empty

Solution Strategy:
A Linked Queue applies elements universally allocating at the tail bounds via tracking explicit ``rear->next = nn``

operations avoiding arbitrary list evaluation traversals completely bounding insertion tasks inside raw $O(1)$ runtime executions simultaneously alongside native generic `front` bounds node deletions.

Reference Solution:


```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* front = NULL;
Node* rear = NULL;

void enqueue(int v) {
    Node* nn = (Node*)malloc(sizeof(Node));
    nn->data = v;
    nn->next = NULL;
    if (rear) rear->next = nn;
    else front = nn;
    rear = nn;
}

void dequeue() {
    if (!front) { printf("Underflow\n"); return; }
    Node* t = front;
    front = front->next;
    if (!front) rear = NULL;
    free(t);
}

void get_front() {
    if (!front) { printf("Empty\n"); return; }
    printf("%d\n", front->data);
}

void display() {
    if (!front) { printf("Empty\n"); return; }
    Node* t = front;
    while (t) {
        printf("%d%c", t->data, t->next ? ' ' : '\n');
        t = t->next;
    }
}

int main() {
    int N, v;
    char op[20];
    if (scanf("%d", &N) != 1) return 0;
    while (N--) {
        scanf("%s", op);
        if (!strcmp(op, "enqueue")) { scanf("%d", &v); enqueue(v); }
        else if (!strcmp(op, "dequeue")) dequeue();
        else if (!strcmp(op, "front")) get_front();
        else if (!strcmp(op, "display")) display();
    }
    return 0;
}

```

Week 7: Doubly-Linked List

Q1. Doubly Linked List Operations - Insertion

MEDIUM CODING

Implement a Doubly Linked List supporting creation, insertion at the beginning, middle, and end, as well as displaying the list.

Input Format: Integer N (number of operations), followed by N lines of commands: 'ins_beg X', 'ins_mid pos X', 'ins_end X', 'display'.

Output Format: For 'display', print space-separated elements. Print 'Empty' if empty or 'Invalid' for bad insertions.

Constraints: $1 \leq N \leq 1000$

Hints: Maintain 'head' and 'tail' pointers to allow $O(1)$ insertions at the end. For middle insertions, traverse to pos-1.

Test Cases:

Sample Test Case #1

Input:

```
5
ins_end 10
ins_beg 5
ins_end 20
ins_mid 2 15
display
```

Expected Output:

```
5 15 10 20
```

Sample Test Case #2

Input:

```
3
ins_beg 1
ins_mid 3 2
display
```

Expected Output:

```
Invalid
1
```

Sample Test Case #3

Input:

```
1
display
```

Expected Output:

```
Empty
```

Solution Strategy:

A complete doubly-linked list focused on creation and insertion operations only (beginning, middle, and end).

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node *p, *n; } Node;
Node *h=NULL, *t=NULL;
int sz=0;
void ib(int v) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=h; nn->p=NULL; if(h) h->p=nn; else t=nn; h=nn; sz++; }
void ie(int v) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=NULL; nn->p=t; if(t) t->n=nn; else h=nn; t=nn; sz++; }
void im(int pos, int v) { if(pos<1 || pos>sz+1) { printf("Invalid\n"); return; } if(pos==1) { ib(v); return; } if(pos==sz+1) { ie(v); return; } Node* nn=malloc(sizeof(Node)); nn->d=v; Node* c=h; for(int i=1; i<pos-1; i++) c=c->n; nn->n=c->n; nn->p=c; if(c->n) c->n->p=nn; c->n=nn; sz++; }
void disp() { if(!h) { printf("Empty\n"); return; } Node* c=h; while(c) { printf("%d%c", c->d, c->n?' ':'\n'); c=c->n; } }
int main() { int n, v, p; char op[20]; if(scanf("%d", &n)!=1) return 0; while(n--) { scanf("%s", op); if(!strcmp(op, "ins_beg")) { scanf("%d", &v); ib(v); } else if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); } else if(!strcmp(op, "ins_mid")) { scanf("%d%d", &p, &v); im(p, v); } else if(!strcmp(op, "display")) disp(); } return 0; }
```

Q2. Doubly Linked List Operations - Deletion

MEDIUM CODING

Implement a Doubly Linked List supporting creation, deletion at the beginning, middle, and end, as well as displaying the list.

Input Format: Integer N (number of operations), followed by N lines of commands: 'ins_end X', 'del_beg', 'del_mid pos', 'del_end', 'display'.

Output Format: For 'display', print space-separated elements. Print 'Empty' if empty or 'Underflow'/'Invalid' for bad deletions.

Constraints: $1 \leq N \leq 1000$

Hints: First generate a list using 'ins_end X'. For 'del_mid', carefully connect the previous node's next pointer to the next node, and vice versa.

Test Cases:

Sample Test Case #1

Input:

```
7
ins_end 10
ins_end 20
ins_end 30
display
del_beg
del_end
display
```

Expected Output:

```
10 20 30
20
```

Sample Test Case #2

Input:

```
6
ins_end 1
ins_end 2
ins_end 3
del_mid 2
del_mid 2
display
```

Expected Output:

```
1
```

Sample Test Case #3

Input:

```
3
del_beg
display
del_mid 1
```

Expected Output:

```
Underflow
Empty
Invalid
```

Solution Strategy:

A complete doubly-linked list focusing on deletion limits (beginning, middle, and end). It populates data utilizing a basic end-insertion.

Reference Solution:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct Node { int d; struct Node *p, *n; } Node;
Node *h=NULL, *t=NULL;
int sz=0;
void ie(int v) { Node* nn=malloc(sizeof(Node)); nn->d=v; nn->n=NULL; nn->p=t; if(t) t->n=nn; else h=nn; t=nn; sz++; }
void db() { if(!h) { printf("Underflow\n"); return; } Node* tm=h; h=h->n; if(h) h->p=NULL; else t=NULL; free(tm); sz--; }
void de() { if(!t) { printf("Underflow\n"); return; } Node* tm=t; t=t->p; if(t) t->n=NULL; else h=NULL; free(tm); sz--; }
void dm(int pos) { if(pos<1 || pos>sz) { printf("Invalid\n"); return; } if(pos==1) { db(); return; } if(pos==sz) { de(); return; } Node* c=h; for(int i=1; i<pos; i++) c=c->n; c->p->n=c->n; if(c->n) c->n->p=c->p; free(c); sz--; }
void disp() { if(!h) { printf("Empty\n"); return; } Node* c=h; while(c) { printf("%d%c", c->d, c->n?' ':'\n'); c=c->n; } }
int main() { int n, v, p; char op[20]; if(scanf("%d", &n)!=1) return 0; while(n--) { scanf("%s", op); if(!strcmp(op, "ins_end")) { scanf("%d", &v); ie(v); } else if(!strcmp(op, "del_beg")) db(); else if(!strcmp(op, "del_end")) de(); else if(!strcmp(op, "del_mid")) { scanf("%d", &p); dm(p); } else if(!strcmp(op, "display")) disp(); } return 0; }

```

Week 9: BST Traversals

Q1. BST Preorder Traversal

EASY CODING

Implement a Binary Search Tree (BST). Read N integers to construct the BST. Print the Preorder traversal of the resulting BST.

Input Format: Integer N, followed by N space-separated integers for insertion.

Output Format: A single line containing space-separated integers for the Preorder traversal. Print 'Empty' if the tree is empty (N=0).

Constraints: $0 \leq N \leq 1000$

Hints: For Preorder traversal, process the root node first, then recursively traverse the left subtree, followed by the right subtree.

Test Cases:

Sample Test Case #1

Input:

5
5 3 7 2 4

Expected Output:

5 3 2 4 7

Sample Test Case #2

Input:

0

Expected Output:

Empty

Sample Test Case #3

Input:

3
1 2 3

Expected Output:

1 2 3

Solution Strategy:

A Node structure is defined, elements are inserted and then Preorder traversal is applied which visits root, left subtree, then right subtree.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int data; struct Node *left, *right; } Node;
Node* insert(Node* root, int val) { if(!root) { Node* nn=malloc(sizeof(Node)); nn->data=val; nn->left=nn->right=NULL; return nn; } if(val<root->data) root->left=insert(root->left, val); else root->right=insert(root->right, val); return root; }
int first=1;
void pre(Node* root) { if(!root) return; printf("%s%d", first?" ":" ", root->data); first=0; pre(root->left); pre(root->right); }
int main() { int n, v; if(scanf("%d", &n)!=1) return 0; Node* root=NULL; for(int i=0; i<n; i++) { scanf("%d", &v); root=insert(root, v); } if(!root) { printf("Empty\n"); return 0; } first=1; pre(root); printf("\n"); return 0; }
```

Q2. BST Inorder Traversal

EASY CODING

Implement a Binary Search Tree (BST). Read N integers to construct the BST. Print the Inorder traversal of the resulting BST.

Input Format: Integer N, followed by N space-separated integers for insertion.

Output Format: A single line containing space-separated integers for the Inorder traversal. Print 'Empty' if the tree is empty (N=0).

Constraints: $0 \leq N \leq 1000$

Hints: *Inorder traversal of a BST always yields elements in sorted order. Recursively traverse the left subtree, then process root node, then right subtree.*

Test Cases:

Sample Test Case #1

Input:

5
5 3 7 2 4

Expected Output:

2 3 4 5 7

Sample Test Case #2

Input:

0

Expected Output:

Empty

Sample Test Case #3

Input:

3
1 2 3

Expected Output:

1 2 3

Solution Strategy:

Elements are inserted comparing values and the Inorder traversal is applied visiting left subtree, root, right subtree respectively.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int data; struct Node *left, *right; } Node;
Node* insert(Node* root, int val) { if(!root) { Node* nn=malloc(sizeof(Node)); nn->data=val; nn->left=nn->right=NULL; return nn; } if(val<root->data) root->left=insert(root->left, val); else root->right=insert(root->right, val); return root; }
int first=1;
void in(Node* root) { if(!root) return; in(root->left); printf("%s%d", first?"":" ", root->data); first=0; in(root->right); }
int main() { int n, v; if(scanf("%d", &n)!=1) return 0; Node* root=NULL; for(int i=0; i<n; i++) { scanf("%d", &v); root=insert(root, v); } if(!root) { printf("Empty\n"); return 0; } first=1; in(root); printf("\n"); return 0; }
```

Q3. BST Postorder Traversal

EASY CODING

Implement a Binary Search Tree (BST). Read N integers to construct the BST. Print the Postorder traversal of the resulting BST.

Input Format: Integer N, followed by N space-separated integers for insertion.

Output Format: A single line containing space-separated integers for the Postorder traversal. Print 'Empty' if the tree is empty (N=0).

Constraints: $0 \leq N \leq 1000$

Hints: For Postorder traversal, recursively traverse the left subtree, then the right subtree, and finally process the root node.

Test Cases:

Sample Test Case #1

Input:

5
5 3 7 2 4

Expected Output:

2 4 3 7 5

Sample Test Case #2

Input:

0

Expected Output:

Empty

Sample Test Case #3

Input:

3

1 2 3

Expected Output:

3 2 1

Solution Strategy:

Elements are inserted comparing values and the Postorder traversal visits left subtree, right subtree then root.

Reference Solution:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int data; struct Node *left, *right; } Node;
Node* insert(Node* root, int val) { if(!root) { Node* nn=malloc(sizeof(Node)); nn->data=val; nn->left=nn->right=NULL; return nn; } if(val<root->data) root->left=insert(root->left, val); else root->right=insert(root->right, val); return root; }
int first=1;
void post(Node* root) { if(!root) return; post(root->left); post(root->right); printf("%s%d", first?"": " ", root->data); first=0; }
int main() { int n, v; if(scanf("%d", &n)!=1) return 0; Node* root=NULL; for(int i=0; i<n; i++) { scanf("%d", &v); root=insert(root, v); } if(!root) { printf("Empty\n"); return 0; } first=1; post(root); printf("\n"); return 0; }
```

Week 10: Sorting Techniques I

Q1. Bubble Sort (Intermediate Steps)

EASY CODING

Implement Bubble Sort to sort an array of integers in ascending order. You MUST print the entire array state after each complete pass of the outer loop.

Input Format: Integer N, followed by N space-separated integers.

Output Format: Print the space-separated array on a new line after each pass. For $N \geq 2$, output $N-1$ lines. For $N=0$ output an empty line, and for $N=1$ output a single line with the element.

Constraints: $0 \leq N \leq 1000$

Hints: Compare adjacent elements and swap them if they are in the wrong order. Use an outer loop running $N-1$ times, and print the array at the end of it.

Test Cases:

Sample Test Case #1

Input:

```
5
5 4 3 2 1
```

Expected Output:

```
4 3 2 1 5
3 2 1 4 5
2 1 3 4 5
1 2 3 4 5
```

Sample Test Case #2

Input:

```
0
```

Expected Output:

Sample Test Case #3

Input:

```
1
42
```

Expected Output:

```
42
```

Solution Strategy:

Intermediate state printing prevents algorithm evasion.

Reference Solution:

```

#include <stdio.h>
void pa(int a[], int n) { for(int i=0; i<n; i++) printf("%d%c", a[i], i==n-1?'\\n':' '); }
int main() {
    int n; if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(a[j]>a[j+1]) {
                int t=a[j]; a[j]=a[j+1]; a[j+1]=t;
            }
        }
        pa(a,n);
    }
    return 0;
}

```

Q2. Insertion Sort (Intermediate Steps)

EASY CODING

Implement Insertion Sort to sort an array of integers in ascending order. You MUST print the entire array state after placing each picked element into its sorted position.

Input Format: Integer N, followed by N space-separated integers.

Output Format: Print the space-separated array on a new line after each insertion iteration. For $N \geq 2$, output N-1 lines. For $N=0$ output an empty line, and for $N=1$ output a single line with the element.

Constraints: $0 \leq N \leq 1000$

Hints: *Maintain a sorted sub-array at the beginning. Take the next element, 'insert' it, and print the entire array.*

Test Cases:

Sample Test Case #1

Input:

5
5 4 3 2 1

Expected Output:

4 5 3 2 1
3 4 5 2 1
2 3 4 5 1
1 2 3 4 5

Sample Test Case #2

Input:

0

Expected Output:

Sample Test Case #3

Input:

1

42

Expected Output:

42

Solution Strategy:

Intermediate state printing prevents algorithm evasion.

Reference Solution:

```
#include <stdio.h>
void pa(int a[], int n) { for(int i=0; i<n; i++) printf("%d%c", a[i], i==n-1?'\n':' '); }
int main() {
    int n; if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    for(int i=1; i<n; i++) {
        int k=a[i], j=i-1;
        while(j>=0 && a[j]>k) { a[j+1]=a[j]; j--; }
        a[j+1]=k;
        pa(a,n);
    }
    return 0;
}
```

Q3. Selection Sort (Intermediate Steps)

EASY **CODING**

Implement Selection Sort to sort an array of integers in ascending order. You **MUST** print the entire array state after each minimum-finding and swap pass.

Input Format: Integer N, followed by N space-separated integers.

Output Format: Print the space-separated array on a new line after each outer loop pass. For $N \geq 2$, output $N-1$ lines. For $N=0$ output an empty line, and for $N=1$ output a single line with the element.

Constraints: $0 \leq N \leq 1000$

Hints: Find the minimum element in the unsorted portion, swap it with the first unsorted element, then print the array.

Test Cases:

Sample Test Case #1

Input:

5
5 4 3 2 1

Expected Output:

1 4 3 2 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5

Sample Test Case #2

Input:

0

Expected Output:

Sample Test Case #3

Input:

1
42

Expected Output:

42

Solution Strategy:

Intermediate state printing prevents algorithm evasion.

Reference Solution:

```
#include <stdio.h>
void pa(int a[], int n) { for(int i=0; i<n; i++) printf("%d%c", a[i], i==n-1?'\n':' '); }
int main() {
    int n; if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    for(int i=0; i<n-1; i++) {
        int m=i;
        for(int j=i+1; j<n; j++) if(a[j]<a[m]) m=j;
        int t=a[i]; a[i]=a[m]; a[m]=t;
        pa(a,n);
    }
    return 0;
}
```

Week 11: Sorting Techniques II

Q1. Merge Sort (Intermediate Steps)

MEDIUM

CODING

Implement Merge Sort to sort an array of integers in ascending order. You MUST print the entire array state immediately after every sub-array merge operation finishes.

Input Format: Integer N, followed by N space-separated integers.

Output Format: Print the full space-separated array on a new line after each successful `merge`.

Constraints: $0 \leq N \leq 1000$

Hints: *Divide the array into halves, sort them recursively, and merge. Print the global array at the very end of your `merge` function.*

Test Cases:

Sample Test Case #1

Input:

```
5
5 4 3 2 1
```

Expected Output:

```
4 5 3 2 1
3 4 5 2 1
3 4 5 1 2
1 2 3 4 5
```

Sample Test Case #2

Input:

```
0
```

Expected Output:

Sample Test Case #3

Input:

```
1
42
```

Expected Output:

```
42
```

Solution Strategy:

Merge sort relies on printing after the merge step is completely finished, leaving an exact execution fingerprint.

Reference Solution:

```

#include <stdio.h>
int n;
void pa(int a[], int n) { for(int i=0; i<n; i++) printf("%d%c", a[i], i==n-1?'\\n':' '); }
void merge(int arr[], int l, int m, int r) {
    int i, j, k; int n1 = m - l + 1; int n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; } k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
    pa(arr, n);
}
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
int main() {
    if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    mergeSort(a, 0, n - 1);
    return 0;
}

```

Q2. Quick Sort (Intermediate Steps)

MEDIUM

CODING

Implement Quick Sort to sort an array of integers in ascending order. You MUST print the entire array state immediately after every successful partition operation.

Input Format: Integer N, followed by N space-separated integers.

Output Format: Print the full space-separated array on a new line after each `partition`.

Constraints: $0 \leq N \leq 1000$

Hints: *Pick a pivot (last element), partition the array, and IMMEDIATELY print the global array state before recursing into the lower and upper halves.*

Test Cases:

Sample Test Case #1

Input:

5

5 4 3 2 1

Expected Output:

1 4 3 2 5

1 4 3 2 5

1 2 3 4 5

1 2 3 4 5

Sample Test Case #2

Input:

0

Expected Output:

Sample Test Case #3

Input:

1

42

Expected Output:

42

Solution Strategy:

Printing after the partition routine locks the student into a strict subset of Quick Sort algorithm steps.

Reference Solution:


```

#include <stdio.h>
int n;
void pa(int a[], int n) { for(int i=0; i<n; i++) printf("%d%c", a[i], i==n-1?'\\n':' '); }
void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) { i++; swap(&arr[i], &arr[j]); }
    }
    swap(&arr[i + 1], &arr[high]);
    pa(arr, n);
    return (i + 1);
}
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main() {
    if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    quickSort(a, 0, n - 1);
    return 0;
}

```

Week 12: Sorting Techniques III

Q1. Heap Sort (Intermediate Steps)

HARD **CODING**

Implement Heap Sort to sort an array of integers in ascending order. You MUST print the initial fully built max heap array in one line, and then print the array state after EACH element extraction.

Input Format: Integer N, followed by N space-separated integers.

Output Format: Print the max heap structure first. Then print the array after every max element is bubbled to the end.

Constraints: $0 \leq N \leq 1000$

Hints: Build a Max Heap and print it. Then repeatedly extract the maximum element (root), swap it with the last element of the remaining heap, re-heapify, and PRINT the array.

Test Cases:

Sample Test Case #1

Input:

5
5 4 3 2 1

Expected Output:

5 4 3 2 1
4 2 3 1 5
3 2 1 4 5
2 1 3 4 5
1 2 3 4 5

Sample Test Case #2

Input:

0

Expected Output:

Sample Test Case #3

Input:

1
42

Expected Output:

42

Solution Strategy:

Printing after extraction shows the gradual assembly of the sorted subarray at the end of the heap.

Reference Solution:

```

#include <stdio.h>
int n;
void pa(int a[], int n) { for(int i=0; i<n; i++) printf("%d%c", a[i], i==n-1?'\\n':' '); }
void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
void heapify(int arr[], int sz, int i) {
    int largest = i; int l = 2 * i + 1; int r = 2 * i + 2;
    if (l < sz && arr[l] > arr[largest]) largest = l;
    if (r < sz && arr[r] > arr[largest]) largest = r;
    if (largest != i) { swap(&arr[i], &arr[largest]); heapify(arr, sz, largest); }
}
void heapSort(int arr[], int sz) {
    for (int i = sz / 2 - 1; i >= 0; i--) heapify(arr, sz, i);
    pa(arr, n);
    for (int i = sz - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
        pa(arr, n);
    }
}
int main() {
    if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    heapSort(a, n);
    return 0;
}

```

Q2. Radix Sort (Intermediate Steps)

HARD **CODING**

Implement Radix Sort to sort an array of non-negative integers in ascending order. You **MUST** print the entire array state after each digit-place (1s, 10s, 100s, etc.) is fully sorted.

Input Format: Integer N, followed by N space-separated non-negative integers.

Output Format: Print the full array after the 1s place is sorted, then 10s place, etc.

Constraints: $0 \leq N \leq 1000$

Hints: Sort by individual digits starting from the least significant digit to the most significant digit, printing the main array at the very end of your counting sort subroutine.

Test Cases:

Sample Test Case #1

Input:

```

5
170 45 75 90 802

```

Expected Output:

```

170 90 802 45 75
802 45 170 75 90
45 75 90 170 802

```

Sample Test Case #2

Input:

0

Expected Output:

Sample Test Case #3

Input:

1

42

Expected Output:

42

Solution Strategy:

Printing layer by digit ensures students use Radix extraction and counting sort.

Reference Solution:

```

#include <stdio.h>
#include <stdlib.h>
int n;
void pa(int a[], int sz) { for(int i=0; i<sz; i++) printf("%d%c", a[i], i==sz-1?'\n':' '); }
int getMax(int arr[], int sz) { int mx = arr[0]; for (int i = 1; i < sz; i++) if (arr[i] > mx)
    mx = arr[i]; return mx; }
void countSort(int arr[], int sz, int exp) {
    int* output = (int*)malloc(sz * sizeof(int)); int i, count[10] = { 0 };
    for (i = 0; i < sz; i++) count[(arr[i] / exp) % 10]++;
    for (i = 1; i < 10; i++) count[i] += count[i - 1];
    for (i = sz - 1; i >= 0; i--) { output[count[(arr[i] / exp) % 10] - 1] = arr[i]; count[(arr
    [i] / exp) % 10]--; }
    for (i = 0; i < sz; i++) arr[i] = output[i];
    pa(arr, sz);
    free(output);
}
void radixSort(int arr[], int sz) {
    if(sz <= 0) return;
    int m = getMax(arr, sz);
    int exp = 1;
    do {
        countSort(arr, sz, exp);
        exp *= 10;
    } while(m / exp > 0);
}
int main() {
    if(scanf("%d",&n)!=1) return 0;
    if(n==0) { printf("\n"); return 0; }
    int a[1005]; for(int i=0; i<n; i++) scanf("%d", &a[i]);
    if(n==1) { pa(a,n); return 0; }
    radixSort(a, n);
    return 0;
}

```

Week 13: Graph Traversals

Q1. Breadth First Search (BFS)

MEDIUM

CODING

Implement Breadth First Search (BFS) for an undirected graph. Given N vertices, M edges, and a source vertex S , print the BFS traversal.

Input Format: Integers N and M , followed by M lines of edges ' $u\ v$ '. Followed by the source vertex S .

Output Format: Space-separated BFS traversal. Start from S . Visit neighbors in increasing order of their vertex IDs.

Constraints: $0 \leq N \leq 100$, $0 \leq M \leq N*(N-1)$

Hints: Use a queue. Enqueue the starting node, mark as visited. Dequeue, print, enqueue unvisited adjacent nodes in order.

Test Cases:

Sample Test Case #1

Input:

```
4 4
0 1
0 2
1 2
2 3
2
```

Expected Output:

```
2 0 1 3
```

Sample Test Case #2

Input:

```
5 4
0 1
0 2
1 3
1 4
0
```

Expected Output:

```
0 1 2 3 4
```

Sample Test Case #3

Input:

```
3 0
0
```

Expected Output:

```
0
```

Solution Strategy:

BFS systematically visits all nodes connected to S in rings, checking neighbors layer by layer using a Queue.

Reference Solution:

```
#include <stdio.h>
int q[1005], f=0, r=0; void push(int v) { q[r++]=v; } int pop() { return q[f++]; } int empty()
{ return f==r; }
int main() { int n, m, u, v, s; if(scanf("%d%d", &n, &m)!=2) return 0; int g[105][105]={0}, vis
[105]={0}; for(int i=0; i<m; i++) { scanf("%d%d", &u, &v); if(u>=0 && u<n && v>=0 && v<n) {
g[u][v]=1; g[v][u]=1; } } if(scanf("%d", &s)!=1) return 0; if(n==0 || s<0 || s>=n) { print
f("\n"); return 0; } push(s); vis[s]=1; int first=1; while(!empty()) { int c=pop(); printf(
"%s%d", first?" ":" ", c); first=0; for(int i=0; i<n; i++) { if(g[c][i] && !vis[i]) { vis[i]
=1; push(i); } } } printf("\n"); return 0; }
```

Q2. Depth First Search (DFS)

MEDIUM **CODING**

Implement Depth First Search (DFS) for an undirected graph. Given N vertices, M edges, and a source vertex S, print the DFS traversal.

Input Format: Integers N and M, followed by M lines of edges 'u v'. Followed by the source vertex S.

Output Format: Space-separated DFS traversal. Start from S. Visit neighbors in increasing order of their vertex IDs recursively.

Constraints: $0 \leq N \leq 100$, $0 \leq M \leq N*(N-1)$

Hints: Use recursion or a stack. Start at node S, mark it as visited, print it, and recursively perform DFS on its first unvisited adjacent node.

Test Cases:

Sample Test Case #1

Input:

```
4 4
0 1
0 2
1 2
2 3
2
```

Expected Output:

```
2 0 1 3
```

Sample Test Case #2

Input:

```
5 4
0 1
0 2
1 3
1 4
0
```

Expected Output:

```
0 1 3 4 2
```

Sample Test Case #3

Input:

```
3 0
0
```

Expected Output:

```
0
```

Solution Strategy:

DFS explores as far down a path as possible before backtracking. We can achieve this elegantly using the call stack in recursion.

Reference Solution:

```
#include <stdio.h>
int g[105][105], vis[105], n;
int first=1;
void dfs(int u) { vis[u]=1; printf("%s%d", first?"": " ", u); first=0; for(int i=0; i<n; i++) {
    if(g[u][i] && !vis[i]) dfs(i); } }
int main() { int m, u, v, s; if(scanf("%d%d", &n, &m)!=2) return 0; for(int i=0; i<105; i++) for(int j=0; j<105; j++) g[i][j]=0; for(int i=0; i<105; i++) vis[i]=0; for(int i=0; i<m; i++) { scanf("%d%d", &u, &v); if(u>=0 && u<n && v>=0 && v<n) { g[u][v]=1; g[v][u]=1; } } if(scanf("%d", &s)!=1) return 0; if(n==0 || s<0 || s>=n) { printf("\n"); return 0; } dfs(s); printf("\n"); return 0; }
```